Binary Search Tree (BST) Notes

Phase 1

- **Entry Point –** The execution starts at the bottom of the script where the **BinarySearchTree** class is initialized and the **root** variable is set to **None**

```
# Initialize the Tree and the root
bst = BinarySearchTree()
root = None
print("--- Starting BST Modified Testing ---")
## Before running tests
## Iniitialize the BINARYSEARCHTREE and ROOT variable to track the state of the tree throughout the operations
```

Figure 1.0

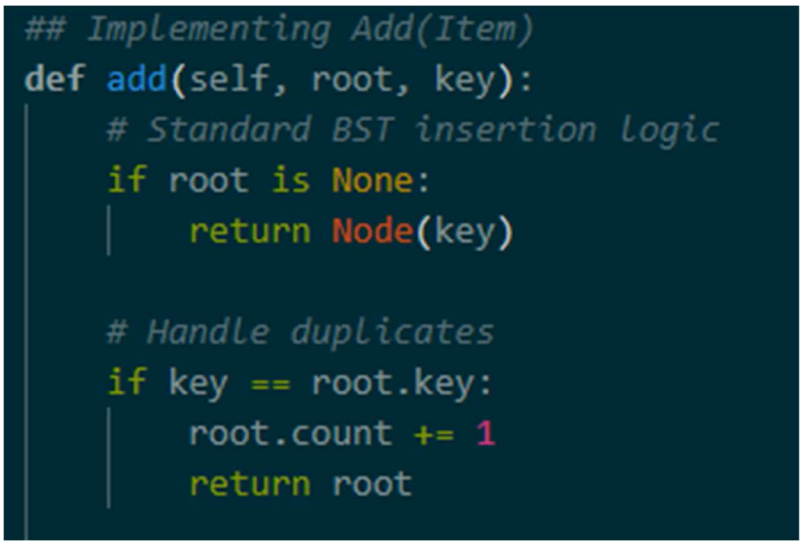
- **Primary Goal –** The goal of this code is to manage a collection of numbers in a tree structure that handles duplicate values and checks for structural balance
- **External Dependencies –** This script is self-contained and does not use external database or APIs. It uses standard Python logic

Phase 2

Step-by-step Workflow

1. **Input & Validation**

- **Action –** Data enters the system through the **add(item)** method
- **Highlight –** The code validates if a key already exists. Instead of adding a new node for the same number, it increments a **count** variable to save space and keep the tree organized



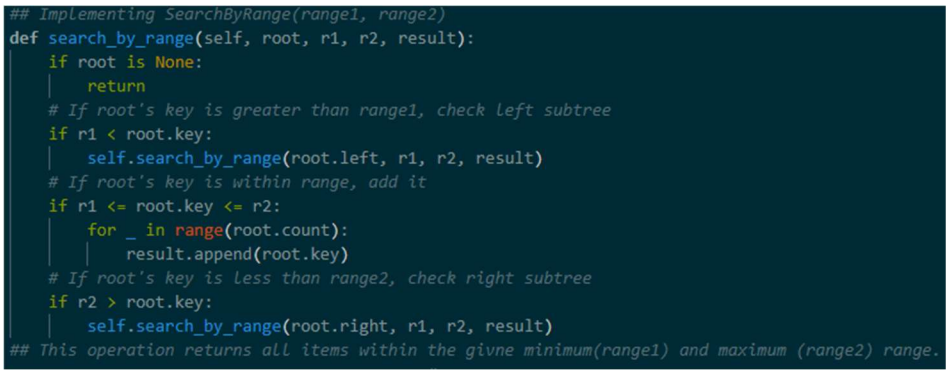
```
## Implementing Add(Item)
def add(self, root, key):
    # Standard BST insertion logic
    if root is None:
        return Node(key)

    # Handle duplicates
    if key == root.key:
        root.count += 1
        return root
```

Figure 1.1

2. **Data Transformation**

- **Action –** The **BinarySearchTree** class contains the main logic for data movement
- **Highlight –**
    - **Recursive Search –** To find, add, or delete items, the code compares the input value to the current node. It moves left if the value is smaller and right if it is larger
    - **Range Filtering –** The **search_by_range** method uses a specific algorithm to visit only the parts of the tree that fall between two numbers



```
## Implementing SearchByRange(range1, range2)
def search_by_range(self, root, r1, r2, result):
    if root is None:
        return
    # If root's key is greater than range1, check left subtree
    if r1 < root.key:
        self.search_by_range(root.left, r1, r2, result)
    # If root's key is within range, add it
    if r1 <= root.key <= r2:
        for _ in range(root.count):
            result.append(root.key)
    # If root's key is less than range2, check right subtree
    if r2 > root.key:
        self.search_by_range(root.right, r1, r2, result)
## This operation returns all items within the givne minimum(range1) and maximum (range2) range.
```

Figure 1.3

3. **Error Handling**

- **Action –** The script uses **if root is None** checks at the beginning of every function
- **Highlight –** If the tree is empty or a search reaches the end of a branch, the code returns **False, None,** or an empty list rather than stopping with an error

Phase 3

Key Highlights

Section	Details
The "Golden Thread"	Data starts as an input in add(), moves to a Node, is stored with a count, and is finally retrieved via search_by_range or removed by delete_min/max.
Potential Bottlenecks	The check_balance_tree function calls get_height repeatedly. In very large trees, this might run slowly because it calculates heights for every single node multiple times.
Data Interaction	The script uses a manual BST structure rather than SQL. It is efficient for searching because it ignores half the tree at every step.
Naming Conventions	Variables like root, lh (left height), and rh (right height) follow standard technical naming patterns.

Personal Reflection

- **Simple Explanation**
    - This code is like an organized filing cabinet. Instead of throwing all papers in one pile, it puts smaller numbers on the left and larger ones on the right so we can find them faster
- **Confusing Part**
    - The most complex part is the **delete_min** and **delete_max** functions because they must remove a node while correctly re-connecting the remaining parts of the tree

---

References

7.13. Search Tree Implementation. (n.d.). Runestone.Academy. Retrieved April 22, 2026, from https://runestone.academy/ns/books/published/pythonds/Trees/SearchTreeImplementation.html

GeeksforGeeks. (2025, July 23). *How to handle duplicates in Binary Search Tree?* GeeksforGeeks. https://www.geeksforgeeks.org/dsa/how-to-handle-duplicates-in-binary-search-tree/

110. Balanced binary tree. (n.d.). AlgoMonster. https://algo.monster/liteproblems/110

Administrator, & Administrator. (2024, August 11). Binary Search Tree Implementation - 101 Computing. *101 Computing - Boost Your Programming Skills!* https://www.101computing.net/binary-search-tree-implementation/